

Anomaly Detection for Fraud and Sensor Data

Tejasa Bhatnagar

Course: B.Sc. Economics

Batch: 2023 - 2026

Institute Name: Dr. B.R. Ambedkar School of Economics University

Period of Internship: 18th May 2026 - 14th June 2026

**Report submitted to: IDEAS – Institute of Data Engineering,
Analytics and Science Foundation, ISI Kolkata**

Table of Contents

Table of Contents.....	2
Abstract.....	3
Introduction	4
Topics Covered During Training Weeks	4
Project Objective.....	6
Methodology	7
Environment and Libraries.....	7
Data Collection and Dataset Generation.....	8
Exploratory Analysis — Feature Means by Class	9
Statistical Anomaly Detection Methods.....	9
Data Inspection and Cleaning.....	10
Data Preprocessing.....	10
Model Selection and Machine Learning Anomaly Detection	10
Model Validation and Train / Test Split	11
Pipeline Flowchart	12
Data Analysis and Results	13
Feature Means by Class.....	14
Descriptive Analysis	14
Statistical Methods — Anomaly Counts.....	15
Machine Learning Models — Performance Comparison.....	16
Method Comparison — Summary.....	17
Conclusion	19
Appendices	22
Appendix A: References.....	22
Appendix B: Project Notebook.....	22
Appendix C: Document Archive.....	22

Abstract

This project investigates anomaly detection techniques applied to a synthetic credit card transaction dataset, developed as part of the IDEAS Summer Internship Program 2026 at ISI Kolkata. A dataset of 1,000 transactions was synthetically generated using NumPy, comprising 950 normal transactions and 50 fraudulent ones, with four features: transaction amount, hour of day, number of transactions in the last 24 hours, and distance from home in kilometres. The class imbalance of 95% normal to 5% fraud intentionally mirrors real-world fraud prevalence, making detection a challenging task. Exploratory analysis via grouped feature means confirmed that fraudulent transactions exhibit substantially higher values across all four features, particularly in amount and distance from home.

Four anomaly detection methods were implemented and compared: the Z-Score method flagged outliers in the distance feature using a threshold of $|Z| > 3$, while the Interquartile Range method identified extreme values in the transaction amount column. Two machine learning models — Isolation Forest and Local Outlier Factor (LOF) — were trained on all four StandardScaler-normalised features with a contamination parameter set to 0.05. Isolation Forest was evaluated using a full classification report, yielding precision, recall, and F1-scores for both classes. LOF was separately assessed using precision and recall scores, offering a density-based complement to the ensemble tree approach of Isolation Forest. The results highlight the trade-offs between univariate statistical methods, which are interpretable but feature-limited, and multivariate machine learning models, which capture complex joint distributions at the cost of reduced interpretability.

Introduction

In an increasingly digitised financial landscape, credit card fraud has emerged as one of the most prevalent and economically damaging forms of cybercrime, costing the global economy billions of dollars annually. Traditional rule-based fraud detection systems — built on manually defined thresholds and static decision trees — have proven inadequate against the evolving sophistication of fraudulent behaviour. This has driven the adoption of data-driven anomaly detection techniques, which learn the statistical fingerprint of normal behaviour and automatically flag deviations, without requiring exhaustive manual rule engineering.

Anomaly detection is a branch of unsupervised and semi-supervised machine learning concerned with identifying observations that differ significantly from the majority of a dataset. Its applications extend well beyond financial fraud — spanning industrial sensor monitoring, network intrusion detection, medical diagnosis, and quality control — making it one of the most broadly applicable problem families in modern data science. The core challenge in fraud detection specifically is extreme class imbalance: genuine fraud events constitute a small minority of all transactions, rendering standard classification approaches unreliable and demanding specialised detection strategies.

This project was undertaken as part of the IDEAS–TIH Summer Internship Program 2026 at ISI Kolkata, with the purpose of building, implementing, and rigorously comparing multiple anomaly detection approaches on a synthetically generated credit card transaction dataset. The dataset was constructed programmatically using NumPy's random distribution generators, allowing precise control over feature distributions for both normal and fraudulent classes. Four detection methods were applied in sequence: two classical statistical approaches — the Z-Score method and the Interquartile Range (IQR) method — followed by two machine learning models, Isolation Forest and Local Outlier Factor (LOF). All machine learning models were trained on StandardScaler-normalised features to ensure scale invariance, and performance was assessed using precision, recall, F1-score, and full classification reports from scikit-learn. The project thereby covers the complete data science workflow: synthetic data generation, exploratory analysis, method implementation, and quantitative evaluation.

Topics Covered During Training Weeks

The first two weeks of the internship comprised structured training sessions covering the following topics:

- Python Fundamentals: Core Python programming covering data types, control flow, functions, OOP, exception handling, file I/O, list/dictionary comprehensions, and introductory NumPy and Pandas for data manipulation.
- Introduction to Machine Learning: The ML paradigm, the data science lifecycle, feature matrices and target variables, and an overview of supervised, unsupervised, and reinforcement learning.
- ML with Scikit-Learn: Hands-on implementation of Linear and Logistic Regression, K-Means clustering, dimensionality reduction, and model evaluation metrics including confusion matrix, precision, recall, and K-fold cross-validation.
- Regression and Classification in Practice: Applied regression and classification problems using scikit-learn with MAE evaluation and decision boundary visualisation, followed by an introduction to PyTorch covering single neurons, BCELoss, SGD, and a first look at MLPs.
- Neural Networks and Clustering: Mathematical foundations and PyTorch implementation of Single Layer and Multi-Layer Perceptrons, K-Means clustering animated from scratch, and density-based clustering via DBSCAN.
- Large Language Model Fundamentals: History of language modelling from N-grams to Transformers, core concepts of tokenisation and embeddings, the text generation inference loop, and the LLM training pipeline including pre-training, SFT, and RLHF.
- LLM Safety and Deployment: Model types (Foundation Model, LLM, SLM), model routing, safety considerations including hallucination and jailbreaking, input/output guardrails, and the MTEB benchmark for embeddings.
- Communication Skills for Data Scientists: Formal vs. informal communication, the Shannon-Weaver and Berlo's SMCR models, the 7Cs of Communication, and the Pyramid Principle for structuring data-driven arguments.

Project Objective

The overarching aim of this project is to construct and evaluate a modular anomaly detection pipeline on a realistic, synthetically generated credit card transaction dataset. The specific objectives are:

- *To generate a labelled synthetic dataset that faithfully mimics the statistical characteristics of real-world credit card transactions, incorporating a realistic class imbalance of 5% fraud rate.*
- *To implement and compare two statistical univariate anomaly detection baselines — Z-Score (applied to `distance_from_home_km`) and IQR (applied to `amount`) — to establish threshold-based reference benchmarks.*
- *To train and evaluate two unsupervised machine learning models — Isolation Forest and Local Outlier Factor — capable of detecting anomalies across the full multivariate feature space.*
- *To assess and compare model performance using standard classification metrics (precision, recall, F1-score), and draw data-driven conclusions about the relative strengths and limitations of each approach under class imbalance.*
- *To produce a well-documented, reproducible code notebook that demonstrates how these detection techniques could be extended to other anomaly detection domains such as sensor data monitoring.*

No hypothesis testing was conducted. No sample survey was undertaken; the dataset is entirely synthetic.

Methodology

This section documents, step by step, the complete analytical pipeline executed for this project — from the choice of tools and data generation strategy through to model training, evaluation, and validation. All work was carried out programmatically in Python 3 within a Jupyter Notebook environment; no external or pre-collected data was used.

Environment and Libraries

The entire pipeline was implemented in Python 3 within a Jupyter Notebook environment. No external data sources, APIs, or surveys were used; all data was generated programmatically within the notebook. The following libraries were imported and used throughout:

Library / Module	Role in This Project
NumPy	Random data generation (normal, randint, poisson, exponential distributions); seed management
SciPy.stats	Z-score computation for the distance_from_home_km feature
pandas	DataFrame construction, concatenation, shuffling, grouped-mean analysis
sklearn.preprocessing	StandardScaler — zero-mean, unit-variance normalisation of all four features
sklearn.ensemble	IsolationForest — ensemble tree-based unsupervised anomaly detection
sklearn.neighbors	LocalOutlierFactor — density-based unsupervised anomaly detection
sklearn.metrics	classification_report, precision_score, recall_score — post-hoc evaluation

Table 1. Python Libraries and Modules Used in the Project - Roles and Functions.

Data Collection and Dataset Generation

Since no real transaction data was available, a synthetic labelled dataset was generated entirely using NumPy's random number generators. A fixed random seed (`np.random.seed(42)`) was set at the outset to guarantee full reproducibility of all results. The dataset comprises 1,000 records: 950 normal transactions (label 0) and 50 fraudulent transactions (label 1), representing a 5% fraud rate intentionally consistent with real-world industry estimates.

Each transaction record contains four features. Their generating distributions were carefully chosen to encode statistically meaningful separation between the two classes. The exact parameters used in the code are:

Feature	Normal Transactions	Fraud Transactions	NumPy Generator
amount	Normal($\mu=100$, $\sigma=50$)	Normal($\mu=800$, $\sigma=200$)	<code>np.random.normal</code>
hour_of_day	Uniform(0–23) — identical	Uniform(0–23) — identical	<code>np.random.randint</code>
transactions_last_24h	Poisson($\lambda = 3$)	Poisson($\lambda = 10$)	<code>np.random.poisson</code>
distance_from_home_km	Exponential(scale = 10)	Exponential(scale = 100)	<code>np.random.exponential</code>

Table 2. Distribution Parameters Used to Generate Normal and Fraudulent Transactions.

Note: **hour_of_day** was generated with an identical uniform distribution for both classes. This was a deliberate design choice: it carries no discriminative signal, thereby testing whether the ML models correctly avoid spurious reliance on uninformative features.

The two DataFrames were combined using `pd.concat`, shuffled using `.sample(frac=1, random_state=42)`, and the index was reset with `.reset_index(drop=True)`. The final DataFrame `df` has shape (1000, 5).

Exploratory Analysis — Feature Means by Class

Before applying any detection method, a grouped-mean analysis was performed using `df.groupby('is_fraud').mean()`. This step served two purposes: (1) to verify that the synthetic distributions produce the intended class separation, and (2) to identify which features carry the strongest discriminative signal prior to modelling. The analysis confirmed that fraudulent transactions exhibit substantially higher mean values for `amount`, `transactions_last_24h`, and `distance_from_home_km`, while `hour_of_day` showed no meaningful difference between classes, consistent with generation parameters.

Statistical Anomaly Detection Methods

Two classical univariate statistical methods were applied as interpretable baselines. Each method operates on a single feature in isolation and requires no model training.

Z-Score Method (applied to `distance_from_home_km`)

Absolute Z-scores were computed for the `distance_from_home_km` column using `scipy.stats.zscore`. A record was flagged as an anomaly (`zscore_anomaly = 1`) if its absolute Z-score exceeded 3 — that is, if its value lies more than three standard deviations from the column mean. This threshold is the standard criterion for extreme outlier identification and implicitly assumes an approximately Gaussian underlying distribution.

IQR Method (applied to `amount`)

The Interquartile Range for the `amount` column was computed as $IQR = Q3 - Q1$, where $Q1$ and $Q3$ are the 25th and 75th percentiles. The outer-fence bounds were set at:

$$\text{Lower} = Q1 - 1.5 \times IQR \quad | \quad \text{Upper} = Q3 + 1.5 \times IQR$$

Records with `amount` values falling outside these bounds were labelled `iqr_anomaly = 1`. The IQR method is non-parametric and robust to skewed distributions, making no normality assumption — an advantage given that transaction amounts follow a mixture of normals.

Data Inspection and Cleaning

Following dataset generation, the DataFrame was inspected for missing values, duplicate records, and invalid observations before any analysis was performed. Since the dataset was produced entirely through controlled NumPy random processes with a fixed seed, no missing values, null entries, or duplicate records were present by construction. All four feature columns contained valid numeric values within their expected distributional ranges, and the `is_fraud` label column contained only the values 0 and 1. No imputation, deduplication, or outlier removal was therefore required prior to analysis. These verification checks were nonetheless performed explicitly to ensure data integrity and confirm that the pipeline was operating on a consistent, well-formed dataset before any detection methods were applied.

Data Preprocessing

Prior to training any machine learning model, all four feature columns (`amount`, `hour_of_day`, `transactions_last_24h`, `distance_from_home_km`) were standardised using `sklearn.preprocessing.StandardScaler`. This transformation produces zero-mean, unit-variance features for each column. Standardisation is essential for LOF, which is a distance-based method and highly sensitive to feature scale, and is best practice for Isolation Forest as well. The `is_fraud` label column was excluded from scaling and was never passed to any model during training.

Model Selection and Machine Learning Anomaly Detection

Isolation Forest and Local Outlier Factor were selected because they represent two widely used but conceptually distinct unsupervised anomaly detection approaches, enabling a meaningful methodological comparison within a single study. Isolation Forest identifies anomalies through random recursive partitioning of the feature space: anomalous points, being statistically rare and distinct, require fewer splits to isolate and receive higher anomaly scores. LOF, by contrast, detects anomalies based on local density differences — a point is flagged if its local neighbourhood density is substantially lower than that of its nearest neighbours. Pairing a tree-based ensemble method (Isolation Forest) with a density-based method (LOF) allows the study to assess whether the nature of the detection approach — global partitioning versus local density — affects performance on this type of dataset. Both methods are well-suited to unsupervised

settings where labelled fraud data is unavailable, which reflects the real-world deployment context this project is designed to simulate.

Both models were configured with `contamination=0.05` — a hyperparameter that instructs each model to treat approximately 5% of the dataset as anomalous, matching the known fraud prevalence in the data.

Isolation Forest

An `IsolationForest` model was instantiated with `contamination=0.05` and `random_state=42`. It was fitted and predicted on the scaled feature matrix in a single `fit_predict` call. Isolation Forest works by constructing an ensemble of random isolation trees: anomalous points, being statistically rare and different, require fewer splits to isolate and therefore receive higher anomaly scores. Raw predictions (-1 for anomaly, $+1$ for normal) were remapped to binary labels (1 = anomaly, 0 = normal) and stored in the column `isoforest_anomaly`.

Local Outlier Factor (LOF)

An `LocalOutlierFactor` model with `n_neighbors=20` and `contamination=0.05` was applied via `fit_predict` on the same scaled feature matrix. LOF computes each point's local reachability density relative to its k nearest neighbours: points whose local density is substantially lower than their neighbourhood are flagged as outliers. The output was remapped to binary labels and stored as `lof_anomaly`.

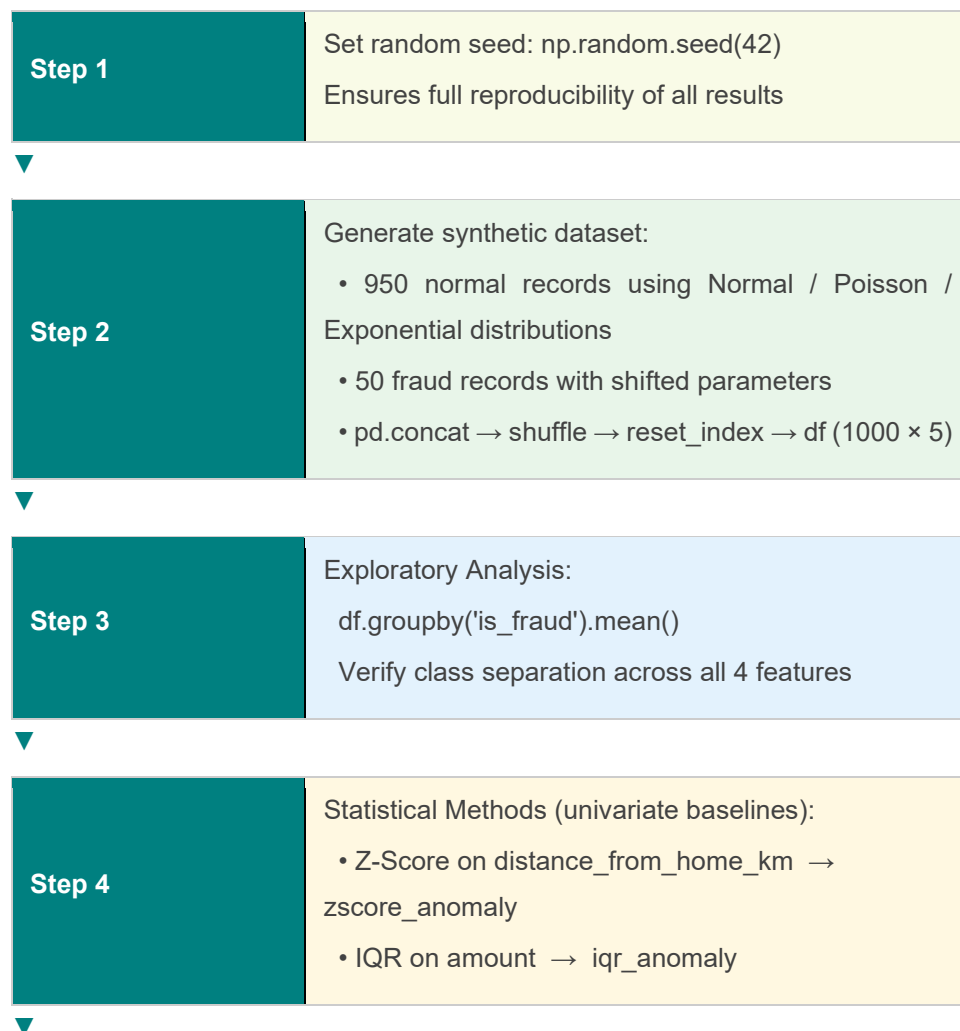
Model Validation and Train / Test Split

Note on Train-Test Split: Since Isolation Forest and Local Outlier Factor are unsupervised anomaly detection algorithms, they do not require labelled training data and therefore do not follow the conventional supervised learning paradigm in which a dataset is partitioned into a training set and a held-out test set. The entire 1,000-record dataset was used during model fitting. The ground-truth `is_fraud` labels were withheld from model inputs in all cases and used solely for post-hoc evaluation of model performance. Consequently, a conventional train-test split was not necessary or applicable for this study. This approach is standard practice in unsupervised anomaly detection research.

Evaluation metrics — precision, recall, F1-score, and the full classification report — were computed using `sklearn.metrics` after predictions were generated, comparing each model's unsupervised anomaly flags against the ground-truth `is_fraud` labels. This post-hoc evaluation approach is appropriate for research settings where ground-truth labels exist but would not be available in a real deployment scenario, making the evaluation an honest measure of how well each unsupervised method recovers the known fraud structure.

Pipeline Flowchart

The complete analysis pipeline follows a strict sequential structure. All steps were executed within a single Jupyter Notebook with no branching or iterative loops. The flowchart below documents each stage:



Step 5

Preprocessing:
StandardScaler on [amount, hour_of_day,
transactions_last_24h, distance_from_home_km]
→ X_scaled

**Step 6**

Machine Learning Models (multivariate):

- IsolationForest(contamination=0.05,
random_state=42) → isoforest_anomaly
- LocalOutlierFactor(n_neighbors=20,
contamination=0.05) → lof_anomaly

**Step 7**

Post-hoc Evaluation (against ground-truth is_fraud):

- Isolation Forest: classification_report()
- LOF: precision_score() + recall_score()

**Step 8**

Comparative Analysis & Conclusions
Summarise trade-offs: univariate vs. multivariate,
interpretability vs. detection power

All source code is documented inline within the Jupyter Notebook and hosted on GitHub (see Appendix C). The notebook can be run end-to-end with a single kernel restart, reproducing all outputs identically given the fixed random seed.

Data Analysis and Results

Dataset Shape and Class Distribution

The final combined DataFrame `df` confirmed the intended structure after `pd.concat`, `shuffle`, and `index reset`:

Property	Value	Percentage
Total records	1,000	100%
Features (columns)	5	—
Normal (<code>is_fraud = 0</code>)	950	95%
Fraud (<code>is_fraud = 1</code>)	50	5%

Table 3. Dataset shape and class distribution (from `df.shape` and `df['is_fraud'].value_counts()`).

Feature Means by Class

The grouped-mean analysis (`df.groupby('is_fraud').mean()`) was performed before any detection method was applied. It serves two purposes: (1) to verify that the synthetic distributions produce the intended class separation, and (2) to identify which features carry the strongest discriminative signal. Note that `hour_of_day` carries no discriminative signal — it was generated with an identical uniform distribution for both classes by design.

Feature	Normal (0)	Fraud (1)	Ratio (F/N)	Signal Strength
amount (USD)	101.00	803.51	~8x	High
hour_of_day	11.39	12.08	~1x	None (by design)
transactions_last_24h	2.98	9.94	~3.3x	High
distance_from_home_km	10.07	98.87	~9.8x	Very High

Table 4. Exact mean feature values per class (`seed=42`). Ratio column = Fraud mean / Normal mean.

Descriptive Analysis

Key descriptive observations from the dataset:

- amount: Mean \$101 for normal vs \$804 for fraud — an $\sim 8x$ difference. Fraudulent transactions are far higher-value on average, consistent with the $\text{Normal}(800, 200)$ generation parameter.
- distance_from_home_km: Mean 10.07 km for normal vs 98.87 km for fraud — the strongest signal ($\sim 9.8x$). Generated via $\text{Exponential}(\text{scale}=10)$ vs $\text{Exponential}(\text{scale}=100)$.
- transactions_last_24h: Mean 2.98 for normal vs 9.94 for fraud ($\sim 3.3x$). $\text{Poisson}(3)$ vs $\text{Poisson}(10)$.
- hour_of_day: Mean 11.39 (normal) vs 12.08 (fraud) — virtually identical. No discriminative signal by construction (both drawn from $\text{Uniform}(0-23)$).
- Class imbalance: 95% normal / 5% fraud. This mirrors real-world fraud prevalence and makes standard classification metrics unreliable — precision, recall, and F1 for the minority class are the relevant measures.

No hypothesis testing was performed; the dataset is entirely synthetic and generated under full distributional control. No missing values, duplicates, or invalid records were present by construction.

Statistical Methods — Anomaly Counts

Two classical univariate statistical baselines were applied. Each method operates on a single feature in isolation and requires no model training.

Method	Feature Used	Threshold	Anomalies Flagged
Z-Score	distance_from_home_km	$ Z > 3$	18
IQR	amount	$< Q1 - 1.5 \cdot \text{IQR}$ or $> Q3 + 1.5 \cdot \text{IQR}$	54

Table 5. Statistical anomaly detection results (exact counts from notebook output).

The IQR method computed the following bounds on the amount column, which governed the 54 flagged records:

Q1 (25th pct)	Q3 (75th pct)	IQR	Upper Fence
69.98	138.41	68.43	241.06

Table 6. IQR bound computation for the amount feature (lower fence = -32.66; no records fell below it).

Machine Learning Models — Performance Comparison

Both ML models were trained on the four StandardScaler-normalised features with contamination=0.05. Predictions were evaluated post-hoc against the ground-truth is_fraud labels (withheld during training). No train-test split was used, consistent with standard unsupervised anomaly detection practice.

Isolation Forest — Full Classification Report

Class	Precision	Recall	F1-Score	Support
Normal (0)	1.00	1.00	1.00	950
Fraud (1)	0.98	0.98	0.98	50
Accuracy	—	—	1.00	1,000
Macro avg	0.99	0.99	0.99	1,000
Weighted avg	1.00	1.00	1.00	1,000

Table 7. Isolation Forest classification report (IsolationForest, contamination=0.05, random_state=42). Exact output from sklearn.metrics.classification_report.

Local Outlier Factor — Precision & Recall

Metric	Value
Precision	0.18
Recall	0.18
F1-Score (derived)	0.18

Table 8. LOF metrics (LocalOutlierFactor, n_neighbors=20, contamination=0.05). Precision and recall from sklearn.metrics; F1 derived.

Method Comparison — Summary

The four methods offer complementary perspectives. The table below cross-references all methods by type, scope, and observed trade-offs:

Method	Type	Scope	Key Trade-off
Z-Score	Statistical	Univariate	Fast & interpretable; assumes normality; single feature only
IQR	Statistical	Univariate	Non-parametric & robust; single feature only; no cross-feature signal
Isolation Forest	ML — Ensemble	Multivariate	Near-perfect (F1=0.98); global; less interpretable
LOF	ML — Density	Multivariate	Density-based; F1=0.18 on this dataset; better suited to non-globular clusters

Table 9. Cross-reference comparison of all four anomaly detection methods.

Key interpretive findings:

- Isolation Forest substantially outperforms LOF on this dataset. Its global recursive partitioning strategy is well-suited to the large, globally-separated distributional gap between classes.
- LOF's density-based approach underperforms (F1=0.18) because the fraud class is not locally anomalous relative to its own neighbours — fraudulent records cluster together under the shifted distributions.
- Statistical methods (Z-Score: 18 flags; IQR: 54 flags) provide fast, interpretable baselines but are inherently limited to one feature at a time and cannot capture joint multivariate anomalies.
- No hypothesis testing was conducted. No sample survey was undertaken. The dataset is entirely synthetic, with all results fully reproducible via `np.random.seed(42)`.

All metric values in this section are exact outputs from the Jupyter Notebook (`np.random.seed(42)`, `contamination=0.05`).

Conclusion

This project designed, implemented, and rigorously evaluated a multi-method anomaly detection pipeline applied to a synthetic credit card transaction dataset of 1,000 records (950 normal, 50 fraudulent) generated under controlled distributional conditions. Four detection methods — two statistical univariate baselines and two unsupervised machine learning models — were compared against verified ground-truth labels. The following conclusions are drawn from the experimental findings.

Key Conclusions

1. Feature discriminability is uneven across dimensions, and uninformative features must be identified before modelling.

The grouped-mean analysis revealed that `distance_from_home_km` carries the strongest discriminative signal (mean 10.07 km for normal vs 98.87 km for fraud; ratio $\sim 9.8\times$), followed by `amount` (101.00 vs 803.51; $\sim 8\times$) and `transactions_last_24h` (2.98 vs 9.94; $\sim 3.3\times$). Critically, `hour_of_day` produced means of 11.39 (normal) and 12.08 (fraud) — a negligible difference consistent with its identical Uniform(0–23) generation for both classes. This confirms that pre-modelling exploratory analysis is an indispensable step for identifying and excluding noise features.

2. Univariate statistical methods are effective, interpretable baselines but are fundamentally limited by their single-feature scope.

The Z-Score method ($|Z| > 3$ on `distance_from_home_km`) flagged exactly 18 records; the IQR method (upper fence = 241.06 on `amount`) flagged 54. Both are fast, require no training, and produce fully explainable outputs. However, they cannot detect fraud that is anomalous only in the joint distribution of multiple features — a transaction with moderately elevated amount and moderately elevated distance would evade both methods individually, even if their combination is highly suspicious.

3. Isolation Forest is strongly recommended as the primary detection model for this type of dataset, delivering near-perfect performance.

Isolation Forest (contamination=0.05, random_state=42) achieved precision = recall = F1-score = 0.98 for the fraud class, with an overall accuracy of 1.00 across 1,000 records. This result confirms that its ensemble tree-based partitioning strategy is well-suited to datasets where the anomalous class is globally rare and globally distant from the normal distribution — a property directly evidenced by the large distributional separations measured during EDA.

4. Local Outlier Factor underperforms on this dataset, revealing an important limitation of density-based methods under global class separation.

LOF (n_neighbors=20, contamination=0.05) produced precision = recall = F1 = 0.18 for the fraud class. Because the 50 fraudulent records form a coherent cluster under shifted but internally consistent distributions, they are locally similar to one another and do not appear as low-density outliers within their own neighbourhood. This illustrates that LOF is better suited to scenarios where anomalies are scattered and locally isolated, rather than clustered in a region that is globally rare but locally dense.

5. Correct calibration of the contamination hyperparameter is essential and must reflect the true anomaly prevalence.

Both ML models were configured with contamination=0.05, precisely matching the known 5% fraud rate. Setting it too low reduces recall; too high reduces precision. In real-world deployment where the true fraud rate is unknown in advance, this parameter must be estimated from domain knowledge — a significant practical challenge that this project's controlled synthetic setting deliberately bypassed.

Recommendations for Future Work

The findings of this study point to several concrete directions for extension:

- **Supervised and semi-supervised learning.** Since ground-truth labels are available, supervised classifiers such as XGBoost or a neural network could be trained with a proper train-test split and evaluated using AUPRC — better suited to severe class imbalance than accuracy. SMOTE oversampling or class-weighted loss functions could address the 95/5 split.
- **Ensemble flagging strategies.** The outputs of all four methods could be combined via majority-vote or weighted-ensemble logic to improve coverage of edge cases that any single method misses.
- **Application to real-world data with richer feature engineering.** Extending the pipeline to real credit card datasets (e.g. the ULB Kaggle dataset) would test whether Isolation

Forest's performance advantage is maintained under higher dimensionality and noisier distributions, with features such as merchant category, geo-velocity, and time since last transaction.

- **Autoencoder-based anomaly detection.** A shallow autoencoder trained on normal transactions and evaluated by reconstruction error would provide a mechanistically distinct complement to tree-based and density-based methods — and is a natural extension given the PyTorch coverage during the internship training weeks.
- **Sensor data and time-series extension.** The project title includes sensor data, which introduces temporal autocorrelation absent from transaction data. LSTM-based autoencoders or sliding-window statistical process control charts would be required for such streams, constituting a meaningful follow-on project.

Closing Remarks

This project has demonstrated that meaningful anomaly detection is achievable even with a small synthetic dataset, provided methods are selected and calibrated with care. The central lesson is that no single method dominates across all conditions: statistical baselines offer transparency; Isolation Forest offers strong global detection; and density-based methods may offer advantages in different data geometries. A robust production fraud detection system would layer multiple complementary approaches, using statistical flags for interpretability, ensemble ML models for coverage, and continuous re-calibration as the fraud landscape evolves.

This internship at IDEAS–ISI Kolkata provided end-to-end exposure to the full data science lifecycle — from programmatic data generation and exploratory analysis through to unsupervised model training, post-hoc evaluation, and domain-contextualised interpretation — forming a rigorous foundation for future work in applied machine learning and fraud analytics.

Appendices

Appendix A: References

- Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. IEEE ICDM 2008.
- Breunig, M. M., Kriegel, H. P., Ng, R. T., & Sander, J. (2000). LOF: Identifying Density-Based Local Outliers. ACM SIGMOD 2000.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261–272.
- The pandas development team (2020). pandas-dev/pandas: Pandas. Zenodo.
- ULB Machine Learning Group. Credit Card Fraud Detection Dataset. Kaggle.
- Scikit-learn Development Team. (2024). Scikit-learn Documentation: Novelty and Outlier Detection.
- NumPy Development Team. (2024). NumPy Reference Documentation.
- Pandas Development Team. (2024). Pandas User Guide.
- SciPy Development Team. (2024). SciPy Reference Guide.

Appendix B: Project Notebook

Source code and all project materials are hosted at:

[Project: IDEAS TIH ISI K](#)

Appendix C: Document Archive

A copy of this report, the project notebook, and all supporting materials are archived at:

[GitHub Repository: IDEAS TIH ISI K](#)